41. Условный тернарный оператор, оператор sizeof и Запятая

На этом уроке мы рассмотрим условный тернарный оператор, оператор Запятую и вспомним оператор sizeof в языке C++.

Оглавление:

1. Оператор sizeof
2. Оператор Запятая
3. Условный тернарный оператор
4. Условный тернарный оператор вычисляется как выражение

Оператор sizeof

Мы уже рассматривали оператор sizeof.

Оператор	Символ	Пример	Операция
sizeof	sizeof	sizeof(тип) sizeof(переменная)	Возвращает размер типа данных или переменной в байтах

Тогда мы использовали его для определения размера конкретных типов данных. Но также sizeof можно использовать и с переменными:

```
#include <iostream>

int main()
{
    double t = 7.0;
    std::cout << sizeof(t); // выводим размер переменной t в байтах
}
```

Оператор Запятая

Оператор Запятая (или ***«оператор Comma»***) позволяет обрабатывать несколько выражений (в то время, когда, обычно, позволяется только одно).

Оператор	Символ	Пример	Операция
Запятая	,	x, y	Вычисляется x, затем вычисляется y, а затем возвращается значение y

Выражение, в котором находится этот оператор, будет иметь значение правого операнда. Например:

```
int x = 0;
int y = 2;
int z = (++x, ++y); // инкремент переменных x и y
```

Переменной `z` будет присвоен результат вычисления `++y` (правого операнда), что равно `3`.

Почти в каждом случае, стейтмент, в котором есть оператор Запятая, лучше записывать в виде отдельных инструкций. Вышеприведенный код корректнее будет записать следующим образом:

```
int x = 0;
int y = 2;
++x;
++y;
int z = y;
```

Обратите внимание, оператор Запятая имеет самый низкий **приоритет** из всех операторов (даже ниже, чем в оператора присваивания), поэтому следующие две строки кода делают не одно и то же:

```
z = (a, b); // сначала вычисляется выражение (a, b), которое равняется значению b, а затем результат присваивается переменной z
z = a, b; // вычисляется как "(z = a), b", поэтому переменной z присваивается значение a, переменная b - игнорируется
```

Большинство программистов не используют оператор Comma вообще (разве что только в **циклах for**).

Обратите внимание, запятая, которая используется в вызовах функций, не является оператором Comma:

```
int sum = add(x, y); // эта запятая не является оператором Comma
```

Аналогично, при объявлении нескольких переменных в одной строке, запятая используется как разделитель, а не как оператор:

```
int x(3), y(5); // эта запятая не является оператором Comma
```

Правило: Избегайте использования оператора Comma (исключением являются циклы for).

Условный тернарный оператор

Условный (тернарный) оператор (обозначается как `?:`) является единственным тернарным оператором в языке C++, который работает с 3-мя операндами. Из-за этого его часто называют просто ***«тернарный оператор»***.

Оператор	Символ	Пример	Операция
Условный	?:	c ? x : y	Если c — ненулевое значение (true), то вычисляется x, в противном случае — y

Оператор `?:` предоставляет сокращенный способ (альтернативу) ветвления if/else.

Стейтменты if/else:

```
if (условие)
    выражение;
else
    другое_выражение;
```

Можно записать как:

```
(условие) ? выражение : другое_выражение;
```

Обратите внимание, операнды условного оператора должны быть выражениями (а не стейтментами).

Например, ветвление if/else, которое выглядит следующим образом:

```
if (условие)
    x = значение1;
else
    x = значение2;
```

Можно записать как:

```
x = (условие) ? значение1 : значение2;
```

Большинство программистов предпочитают последний вариант, так как он читабельнее.

Давайте рассмотрим еще один пример. Чтобы определить, какое значение поместить в переменную `larger`, мы можем сделать так:

```
if (x > y)
    larger = x;
else
    larger = y;
```

Или вот так:

```
larger = (x > y) ? x : y;
```

Обычно, часть с условием помещают внутри скобок, чтобы убедиться, что приоритет операций корректно сохранен и так удобнее читать.

Помните, что оператор `?:` имеет очень низкий приоритет, из-за этого его следует записывать в круглых скобках.

Например, для вывода `x` или `y`, мы можем сделать следующее:

```
if (x > y)
    std::cout << x;
else
    std::cout << y;
```

Или с помощью тернарного оператора:

```
std::cout << ((x > y) ? x : y);
```

Давайте рассмотрим, что произойдет, если мы не заключим в скобки весь условный оператор в вышеприведенном случае. Поскольку оператор `<<` имеет более высокий приоритет, чем оператор `?:`, то следующий стейтмент (где мы не заключили весь тернарный оператор в круглые скобки, а только лишь условие):

```
std::cout << (x > y) ? x : y;
```

Будет обрабатываться как:

```
(std::cout << (x > y)) ? x : y;
```

Таким образом, в консольном окне мы увидим `1` (true), если `x > y`, в противном случае — выведется `0` (false).

> ***Совет:*** Всегда заключайте в скобки условную часть тернарного оператора, а лучше весь тернарный оператор.

Условный тернарный оператор — это удобное упрощение ветвления if/else, особенно при присваивании результата переменной или возврате определенного значения. Но его не следует использовать вместо сложных ветвлений if/else, так как в таких случаях читабельность кода резко ухудшается и вероятность возникновения ошибок только растет.

Правило: Используйте условный тернарный оператор только в тривиальных случаях.

Условный тернарный оператор вычисляется как выражение

Стоит отметить, что условный оператор вычисляется как выражение, в то время как ветвление if/else обрабатывается как набор стейтментов. Это означает, что тернарный оператор `?:` может быть использован там, где if/else применить невозможно, например, при инициализации константы:

```
bool inBigClassroom = false;
const int classSize = inBigClassroom ? 30 : 20;
```

Здесь нельзя использовать if/else, так как константы должны быть инициализированы при объявлении, а стейтмент не может быть значением для инициализации.